증명 집계 (Proof Aggregation)

개요

증명 집계는 여러 개의 증명을 하나의 증명으로 결합하여 온체인 검증 비용을 줄이는 핵심 메커니즘입니다. Scroll의 ZKEVM은 다층 집계 구조를 사용하여 청크(Chunk) → 배치(Batch) → 번들(Bundle)로 증명을 계층적으로 집계합니다.

이 문서에서는 증명 집계의 핵심 구성 요소인 다음 내용을 다룹니다:

- Snark-verifier를 통한 증명 검증
- 재귀적 증명 생성 메커니즘
- 증명 압축 (Compression)
- 최종 번들 증명 생성

집계 아키텍처

계층 구조

```
Chunk Proofs (SuperCircuit)

↓ [Compression Layer 1]

Compressed Chunk Proofs

↓ [Batch Aggregation]

Batch Proof (BatchCircuit)

↓ [Compression Layer 2]

Compressed Batch Proof
```

```
↓ [Recursive Aggregation]

Bundle Proof (RecursionCircuit)

↓ [Final Verification]

On-chain Verification
```

주요 컴포넌트

1. **CompressionCircuit**: 개별 증명의 크기를 줄이는 압축 회로
2. **BatchCircuit**: 여러 청크를 하나의 배치로 집계
3. **RecursionCircuit**: 여러 배치를 재귀적으로 번들로 집계

Snark-Verifier 사용

KZG 누산기 (Accumulator)

증명 집계의 핵심은 KZG 누산기를 사용한 증명 축적입니다.

누산기 구조:

```rust
pub struct KzgAccumulator<C, L> {
pub lhs: L::LoadedEcPoint, // Left-hand side EC point
pub rhs: L::LoadedEcPoint, // Right-hand side EC point
}
```

검증 조건:

```
e(lhs, [1]₂ ) = e(rhs, [s]₂ )
```

여기서:

- `lhs`, `rhs` 는 타원곡선 점
- $[1]_2$, $[s]_2$ 는 G2의 생성원과 신뢰 설정 파라미터
- `e` 는 페어링 함수

증명 추출 및 누산

함수: `extract_accumulators_and_proof`

위치: `aggregator/src/core.rs`

이 함수는 여러 SNARK의 증명을 추출하고 하나의 누산기로 결합합니다:

```
pub(crate) fn  extract_accumulators_and_proof(
params: &ParamsKZG<Bn256>,
snarks: &[Snark],
rng: impl Rng + Send,
g2: &G2Affine,
s_g2: &G2Affine,
) -> Result<(KzgAccumulator<G1Affine, NativeLoader>, Vec<u8>), Error>
```

처리 과정:

1. **개별 증명 검증**:

- 각 SNARK의 증명을 Shplonk 프로토콜로 읽음
- `succinct_verify` 를 통해 누산기 생성
- 페어링 체크로 누산기 유효성 검증

2. **누산기 결합**:

- KzgAs (KZG Accumulation Scheme) 사용
- 무작위 선형 결합으로 여러 누산기를 하나로 축적
- `accumulated_ec_pt = ec_pt_1 * 1 + ec_pt_2 * r + ... + ec_pt_n * r^{n-1}`

3. **증명 생성**:

- Poseidon 트랜스크립트 사용
- 최종 누산기와 AS 증명 반환

페어링 체크

함수: `extract_proof_and_instances_with_pairing_check`

위치: `aggregator/src/core.rs`

```
pub fn extract_proof_and_instances_with_pairing_check(
params: &ParamsKZG<Bn256>,
snarks: &[Snark],
rng: impl Rng + Send,
) -> Result<(Vec<u8>, Vec<Fr>), snark_verifier::Error>
```

이 함수는 누산기 추출 후 페어링 체크를 수행하여 증명의 유효성을 보장합니다:

```
let left = Bn256::pairing(&lhs, &params.g2());
let right = Bn256::pairing(&rhs, &params.s_g2());
assert_eq!(left, right, "accumulator check failed");
```

반환값:

- AS 증명 (accumulation scheme proof)
- 누산기 인스턴스 (12개 필드 원소: lhs.x, lhs.y, rhs.x, rhs.y를 limbs로 분해)

증명 압축 (Compression)

CompressionCircuit

목적: 증명 크기를 줄여 후속 집계 단계의 효율성 향상

위치: `aggregator/src/compression/circuit.rs`

압축 프로세스

1. **입력**:
- 원본 SNARK (청크 증명 또는 배치 증명)
- SVK (Succinct Verifying Key)
- 이전 누산기 존재 여부 플래그
2. **회로 동작**:

```
pub fn synthesize(&self, config: Self::Config, mut layouter: impl Layouter<Fr
-> Result<(), Error>
{
// 1. Halo2 Loader 생성
let loader = Halo2Loader::new(ecc_chip, ctx);
// 2. SNARK 집계 (aggregate 함수 사용)
let (assigned_instances, acc) = aggregate::<Kzg<Bn256, Bdfg21>>(
&self.svk,
```

```
&loader,
&[self.snark.clone()],
self.as_proof(),
);
// 3. 공개 입력 구성
// - 새로운 누산기 (12 필드)
// - 원본 SNARK의 공개 입력 (이전 누산기 제외)
instances.extend(flatten_accumulator(acc).iter().map(|assigned| assigned.cell())
let  skip = if  self.has_accumulator { ACC_LEN } else { 0 };
instances.extend(assigned_instances.iter()
.flat_map(|instance_column| instance_column.iter().skip(skip).map(|x| x.cell()))
}
```

3. **출력**:

- 압축된 증명 (더 작은 크기)
- 동일한 공개 입력 (누산기 + 원본 인스턴스)

압축 레이어

2단계 압축:

1. **청크 압축**:

- SuperCircuit 증명 → Compressed Chunk Proof
- 증명 크기 감소로 배치 집계 효율 향상

2. **배치 압축**:

- BatchCircuit 증명 → Compressed Batch Proof

- 재귀 집계 전 증명 크기 최적화

재귀적 증명 (Recursive Proof)

RecursionCircuit

목적: 여러 배치를 재귀적으로 집계하여 최종 번들 증명 생성

위치: `aggregator/src/recursion/circuit.rs`

StateTransition 트레이트

재귀 회로는 상태 전이를 구현하는 회로를 집계합니다:

```
pub trait StateTransition: Sized {
type Input: Clone;
type Circuit: CircuitExt<Fr>;


// 상태 전이 정의
fn state_transition(&self, round: usize) -> Self::Input;
// 상태 표현에 필요한 필드 수
fn num_transition_instance() -> usize;
// 추가 인스턴스 필드 수
fn num_additional_instance() -> usize;
// 인스턴스 레이아웃 인덱스
fn accumulator_indices() -> Vec<usize>;
fn state_prev_indices() -> Vec<usize>;
```

```
fn state_indices() -> Vec<usize>;

fn additional_indices() -> Vec<usize>;

fn propagate_indices() -> Vec<usize>;

}
```

재귀 구조

재귀 정의:

```
RC_Snark(N) := verify(App_Snark(N)) ∧ verify(RC_Snark(N-1))
```

여기서:

- `RC` : RecursionCircuit
- `App` : Application Circuit (BatchCircuit)
- `N` : 재귀 라운드 번호

첫 번째 라운드:

- 더미 SNARK와 첫 번째 배치 SNARK 결합
- 초기 누산기 사용

후속 라운드:

- 이전 재귀 SNARK와 현재 배치 SNARK 결합
- 누산기 전파 및 업데이트

공개 입력 레이아웃

RecursionCircuit의 공개 입력 구조:

```
| accumulator (12) | preprocessed_digest (1) | init_states | final_states |

| propagated_additional_states | additional_states | round (1) |
```

필드 설명:

1. **accumulator** (4 * LIMBS = 12):

- 모든 검증된 SNARK의 누산기 축적
- lhs.x, lhs.y, rhs.x, rhs.y를 limbs로 분해

2. **preprocessed_digest** (1):

- RecursionCircuit 자체를 나타내는 고유 값
- 전처리된 다항식 커밋의 Poseidon 해시

3. **init_states** (num_transition_instance):

- 초기 상태 (L1에서 마지막으로 확정된 상태)
- 첫 라운드: 첫 배치의 prev_state
- 후속 라운드: 이전 재귀의 init_states 전파

4. **final_states** (num_transition_instance):

- 최종 상태 (현재 배치 후 상태)
- 현재 배치의 post_state

5. **propagated_additional_states**:

- 모든 배치에서 동일해야 하는 추가 상태
- 예: chain_id

6. **additional_states**:

- 마지막 배치에서만 내보내는 추가 상태

7. **round** (1):

- 재귀 라운드 번호 (0부터 시작)

재귀 회로 제약 조건

함수: `RecursionCircuit::synthesize`

주요 제약:

1. **전처리 다이제스트 전파**:

```
if round > 0:
PI(preprocessed_digest) = PI_prev(preprocessed_digest)
```

2. **라운드 증가**:

```
if round > 0:
PI(round) = PI_prev(round) + 1
```

3. **초기 상태 전파**:

```
if round > 0:
PI(init_states) = PI_prev(init_states)
else:
PI(init_states) = PI_app(init_states)
```

4. **최종 상태 투명 전달**:

```
PI_app(final_states) = PI(final_states)
```

5. **추가 상태 전달**:

```
PI_app(all_additional_states) = PI(all_additional_states)
```

6. **전파 상태 유지**:

```
if round > 0:
PI_app(propagated_additional_states) = PI_prev(propagated_additional_states)
```

7. **상태 체이닝**:

```
PI_app(init_states) = PI_prev(final_states)
```

동적 검증 (Dynamic Verification)

함수: `dynamic_verify`

위치: `aggregator/src/recursion/common.rs`

이 함수는 임의의 회로를 검증할 수 있도록 전처리 부분을 witness로 로드합니다:

```
pub fn dynamic_verify<'a, PCS>(
svk: &PCS::SuccinctVerifyingKey,
loader: &Rc<Halo2Loader<'a>>,
snark: &SnarkWitness,
preprocessed_digest: Option<AssignedScalar<'a>>,
) -> (Vec<Vec<AssignedScalar<'a>>>, Vec<PCS::Accumulator>)
```

처리 과정:

1. **전처리 다이제스트 검증** (제공된 경우):

- 전처리된 커밋을 witness로 로드
- Poseidon 해시 계산
- 제공된 다이제스트와 일치 확인

2. **인스턴스 로드**:

- SNARK의 공개 입력을 회로에 할당

3. **증명 검증**:

- Poseidon 트랜스크립트 생성
- Plonk 증명 읽기 및 검증
- 누산기 반환

누산기 선택

함수: `select_accumulator`

첫 번째 라운드에서는 기본 누산기를, 후속 라운드에서는 이전 누산기를 선택:

```
fn  select_accumulator<'a>(
loader: &Rc<Halo2Loader<'a>>,
condition: &AssignedValue<Fr>,
lhs: &KzgAccumulator<G1Affine, Rc<Halo2Loader<'a>>>,
rhs: &KzgAccumulator<G1Affine, Rc<Halo2Loader<'a>>>,
) -> Result<KzgAccumulator<G1Affine, Rc<Halo2Loader<'a>>>, Error>
```

동작:

- `condition` 이 true면 `lhs` 선택
- `condition` 이 false면 `rhs` 선택
- ECC 칩의 `select` 연산 사용

누산기 축적

함수: `accumulate`

여러 누산기를 하나로 결합:

```
fn accumulate<'a>(
loader: &Rc<Halo2Loader<'a>>,
accumulators: Vec<KzgAccumulator<G1Affine, Rc<Halo2Loader<'a>>>>,
as_proof: Value<&'_ [u8]>,
) -> KzgAccumulator<G1Affine, Rc<Halo2Loader<'a>>>
```

처리:

1. Poseidon 트랜스크립트 생성
2. AS (Accumulation Scheme) 증명 읽기
3. AS 검증 및 새 누산기 반환

배치 집계 (Batch Aggregation)

BatchCircuit의 역할

위치: `aggregator/src/aggregation/circuit.rs`

BatchCircuit은 여러 청크를 집계하면서 다음을 검증합니다:

1. **청크 연속성**: 상태 루트 체이닝

2. **데이터 무결성**: 해시 일치

3. **Blob 일관성**: EIP-4844 blob 데이터 검증

해시 셀 추출 및 검증

함수: `assign_batch_hashes`

위치: `aggregator/src/core.rs`

```
pub(crate) fn  assign_batch_hashes<const N_SNARKS: usize>(
keccak_config: &KeccakCircuitConfig<Fr>,
rlc_config: &RlcConfig,
layouter: &mut  impl Layouter<Fr>,
challenges: Challenges<Value<Fr>>,
chunks_are_valid: &[bool],
num_valid_chunks: usize,
preimages: &[Vec<u8>],
) -> Result<AssignedBatchHash, Error>
```

처리 단계:

1. **Keccak 테이블 할당**:

- 모든 해시 프리이미지 처리
- batch_hash, chunk_pi_hash, batch_data_hash

2. **조건부 제약**:

- batch_data_hash가 batch_hash에 재사용됨
- chunk 데이터 해시 일치

- 청크 연속성 (상태 루트)
- chain_id 일관성
- 패딩 청크 처리

3. **RLC 계산**:

- Random Linear Combination으로 다중 바이트 압축
- Keccak 테이블 룩업

조건부 제약 함수

함수: `conditional_constraints`

위치: `aggregator/src/core.rs`

이 함수는 배치 집계의 핵심 제약 조건을 구현합니다:

주요 제약:

1. **batch_data_hash 재사용**:

```
batch_hash_preimage[BATCH_DATA_HASH_OFFSET..] == batch_data_hash_output
```

2. **청크 데이터 해시 일치**:

```
for i in 0..N_SNARKS:
if chunk_is_valid[i]:
chunk_pi_hash_preimage[i][CHUNK_DATA_HASH_INDEX..]
== batch_data_hash_preimage[i*32..(i+1)*32]
```

3. **청크 연속성 (상태 루트)**:

```
for i in 0..N_SNARKS-1:
if chunk_is_valid[i+1]:
chunk[i].post_state_root == chunk[i+1].prev_state_root
```

4. **chain_id 일관성**:

```
for i in 0..N_SNARKS-1:
if chunk_is_valid[i+1]:
chunk[i].chain_id == chunk[i+1].chain_id
```

5. **패딩 청크**:

```
for i in 1..N_SNARKS:
if chunks_are_padding[i]:
chunk_pi_hash_rlc[i] == chunk_pi_hash_rlc[i-1]
```

6. **공개 입력 매핑**:

```
PI.parent_state_root = chunks[0].prev_state_root
PI.current_state_root = chunks[N_SNARKS-1].post_state_root
PI.current_withdraw_root = chunks[N_SNARKS-1].withdraw_root
PI.chain_id = chunks[N_SNARKS-1].chain_id
```

최종 증명 생성

전체 파이프라인

```
1. Chunk Proofs (SuperCircuit)
↓
2. Compression Layer 1
- CompressionCircuit으로 각 청크 증명 압축
↓
3. Batch Aggregation
- BatchCircuit으로 N개 청크 집계
- 상태 전이 및 데이터 무결성 검증
↓
4. Compression Layer 2
- CompressionCircuit으로 배치 증명 압축
↓
5. Recursive Aggregation (Round 0)
- RecursionCircuit으로 첫 배치 처리
- 더미 SNARK와 결합
↓
6. Recursive Aggregation (Round 1..N)
- 각 라운드에서 새 배치 추가
- 이전 재귀 SNARK와 결합
↓
7. Final Bundle Proof
```

- 모든 배치를 포함하는 단일 증명
- 온체인 검증 준비

번들 증명 특징

공개 입력:

- 누산기 (12 필드)
- 전처리 다이제스트 (1 필드)
- 초기 상태 루트 (첫 배치의 prev_state)
- 최종 상태 루트 (마지막 배치의 post_state)
- 최종 withdraw 루트
- chain_id
- 라운드 번호

검증 가능성:

- 단일 페어링 체크로 모든 배치 검증
- 상태 전이 체인 무결성 보장
- 데이터 가용성 보장 (blob 일관성)

최적화 기법

1. Shplonk 누산 스킴

장점:

- 여러 다항식 커밋을 효율적으로 축적
- 단일 페어링 체크로 검증

- 증명 크기 최소화

동작:

```
accumulator = Σ(commitment_i * challenge^i)
```

2. Poseidon 해시

사용처:

- 트랜스크립트 생성
- 전처리 다이제스트 계산
- Challenge 생성

장점:

- SNARK-friendly 해시 함수
- 회로 내 효율적 구현
- 작은 제약 조건 수

3. 동적 회로 검증

특징:

- 전처리 부분을 witness로 로드
- 임의의 회로 검증 가능
- 유연한 집계 구조

4. 조건부 제약

구현:

```
conditional_enforce_equal(region, lhs, rhs, condition, offset)
```

효과:

- 패딩 청크 처리
- 동적 배치 크기 지원
- 회로 재사용성 향상

구현 예시

재귀 회로 생성

```
use aggregator::{RecursionCircuit, StateTransition};

// 첫 번째 재귀 라운드
let  recursion_circuit_0 = RecursionCircuit::<BatchCircuit>::new(
&params,
batch_snark_0, // 첫 번째 배치 SNARK
dummy_snark, // 더미 SNARK
rng,
0, // round = 0
);

// 두 번째 재귀 라운드
let  recursion_snark_0 = gen_snark(&params, &pk, recursion_circuit_0, rng);
```

```rust
let recursion_circuit_1 = RecursionCircuit::<BatchCircuit>::new(
&params,
batch_snark_1, // 두 번째 배치 SNARK
recursion_snark_0, // 이전 재귀 SNARK
rng,
1, // round = 1
);


// 최종 번들 증명
let bundle_snark = gen_snark(&params, &pk, recursion_circuit_1, rng);
```

압축 회로 사용

```rust
use aggregator::CompressionCircuit;


// 청크 증명 압축
let compression_circuit = CompressionCircuit::new(
&params,
chunk_snark, // 원본 청크 SNARK
false, // 이전 누산기 없음
rng,
)?;
```

```
let compressed_snark = gen_snark(&params, &pk, compression_circuit, rng);


// 배치 증명 압축
let batch_compression = CompressionCircuit::new(
&params,
batch_snark, // 배치 SNARK
true, // 이전 누산기 있음 (BatchCircuit 출력)
rng,
)?;
```

관련 문서

- 청킹 전략: 멀티 블록 청킹 메커니즘
- 증명 생성: 개별 증명 생성 프로세스
- Batch Circuit: 배치 회로 구조
- State Circuit: 상태 관리 회로

참고 자료

코드 위치:

- `aggregator/src/core.rs` : 누산기 추출 및 페어링 체크
- `aggregator/src/recursion/circuit.rs` : RecursionCircuit 구현
- `aggregator/src/recursion/common.rs` : 동적 검증 함수

- `aggregator/src/compression/circuit.rs` : CompressionCircuit 구현
- `aggregator/src/aggregation/circuit.rs` : BatchCircuit 구현

외부 의존성:

- `snark-verifier` : SNARK 검증 라이브러리
- `snark-verifier-sdk` : 고수준 집계 API
- `halo2-ecc` : 타원곡선 연산 칩
- `halo2-base` : 기본 게이트 및 범위 체크

요약

증명 집계는 Scroll ZKEVM의 확장성을 가능하게 하는 핵심 메커니즘입니다:

1. **Snark-verifier**를 통해 증명을 검증하고 KZG 누산기로 축적
2. **CompressionCircuit**으로 증명 크기를 줄여 효율성 향상
3. **RecursionCircuit**으로 여러 배치를 재귀적으로 집계
4. **StateTransition** 트레이트로 상태 전이 체인 보장
5. **동적 검증**으로 유연한 회로 집계 지원

최종 번들 증명은 단일 페어링 체크로 수백 개의 블록을 검증할 수 있어, 온체인 검증 비용을 획기적으로 줄입니다.